END-SEMESTER PROJECT REPORT — MILESTONE 2

Intelligent Property Price Prediction & Agentic Real Estate Advisory

From Market Analytics to Autonomous Investment Advice

Project	Project 9
Submission	End-Semester (Milestone 2)
Framework	LangGraph + XGBoost + Groq
Deployment	Streamlit Community Cloud
Live URL	`streamlit.app`
GitHub	`github.com/Jag2007/property-price-prediction`

"This application is a student project for educational purposes only.
It does not constitute financial, legal, or investment advice."

April 18, 2026

Contents

1 Introduction and Problem Statement **2**

2 Dataset Description and Feature Engineering **2**
2.1 Dataset Overview . . . 2
2.2 Raw Feature Description . . . 3
2.3 Preprocessing Pipeline . . . 3

3 Milestone 1: Machine Learning Models **3**
3.1 Model Selection . . . 4
3.2 Hyperparameter Configuration . . . 4
3.3 Model Performance Results . . . 4
3.4 Key Performance Drivers . . . 4
3.5 Saved Artifacts . . . 5

4 Milestone 2: Agentic Real Estate Advisory System **5**
4.1 System Overview . . . 5
4.2 Technology Stack and Design Justification . . . 6
4.3 LangGraph State and Graph Architecture . . . 6
4.4 Node-by-Node Description . . . 7
4.5 Three Input Paths . . . 7
4.5.1 Path 1 — Prompt Agent . . . 8
4.5.2 Path 2 — Manual Input . . . 8
4.5.3 Path 3 — CSV Batch Upload . . . 8
4.6 Comparable Property Retrieval . . . 8
4.7 Prompting Strategy and Anti-Hallucination Guardrails . . . 8
4.7.1 Guardrail 1 — Schema-Locked Input Extraction . . . 8
4.7.2 Guardrail 2 — Human Review Before Prediction . . . 9
4.7.3 Guardrail 3 — Grounded Explanation Prompt . . . 9
4.7.4 Guardrail 4 — Output Format Constraint . . . 9
4.7.5 Guardrail 5 — Safe Fallbacks . . . 9
4.8 Structured Advisory Output . . . 9
4.9 Email Notification . . . 10

5 System Architecture **10**
5.1 Architecture Diagram . . . 10
5.2 Repository Structure . . . 11
5.3 End-to-End Data Flow . . . 12
5.4 Streamlit Pages . . . 12

6 Deployment and Reproducibility **12**
6.1 Public Deployment . . . 12
6.2 Local Setup . . . 13
6.3 Environment Variables . . . 13
6.4 Key Dependencies . . . 14

7 Results and Qualitative Examples **14**
7.1 Quantitative Model Performance 14
7.2 Qualitative Prompt Agent Example 14
7.3 Advisory Label Mapping 15

8 Responsible AI and Ethical Considerations **15**
8.1 Financial Disclaimer 15
8.2 Data Transparency 15
8.3 Hallucination Mitigation 15
8.4 Model Limitations 15

9 Conclusion **16**

References **16**

Introduction and Problem Statement

Real estate valuation is one of the most consequential decision-making domains in personal and institutional finance. Buyers, sellers, and investors routinely face the challenge of estimating fair market value and assessing investment potential with incomplete information, rapidly changing market conditions, and inconsistent data sources. Traditional methods rely on manual comparables analysis and agent expertise, both of which are slow, subjective, and difficult to scale.

This project addresses these limitations by building a two-phase AI system:

1. **Milestone 1 — ML-Based Price Prediction**: A classical machine learning pipeline using XGBoost to predict property market price and investment grade from structured property attributes.
2. **Milestone 2 — Agentic Real Estate Advisory**: A LangGraph-orchestrated multi-agent system that accepts natural-language property descriptions, reasons about predicted values, retrieves comparable properties, and produces a structured investment advisory report.

The final system is publicly deployed on Streamlit Community Cloud, supports three input modes (natural-language prompt, manual form, CSV batch upload), and delivers structured advisory output both on-screen and via email notification.

Dataset Description and Feature Engineering

Dataset Overview

The dataset used in this project is a synthetic real estate listing dataset comprising 10,000 property records generated to reflect realistic Indian urban property market characteristics. The dataset was obtained in raw form (`real_estate_raw.csv`) and cleaned into a processed form (`real_estate_clean.csv`).

Table 1: Dataset summary statistics

Attribute	Value
Total records	10,000
Raw features	18 columns
Model input features	18 (after encoding)
Target 1	`Current_Market_Price` (continuous, INR)
Target 2	`Investment_Grade` (ordinal: 0 = Low, 1 = Medium, 2 = High)
Train / Test split	80% / 20%

Raw Feature Description

Table 2: Input features used for model training

Feature	Type	Description
`Total_Square_Footage`	Numeric	Total built-up area in square feet
`Bedrooms`	Integer	Number of bedrooms
`Bathrooms`	Integer	Number of bathrooms
`Age_of_Property`	Integer	Property age in years
`Floor_Number`	Integer	Floor level of the unit
`Furnishing_Status`	Ordinal	Unfurnished (0), Semi-furnished (1), Fully-furnished (2)
`Distance_to_City_Center_km`	Numeric	Distance from city centre in kilometres
`Proximity_to_Public_Transport_km`	Numeric	Distance from nearest public transport
`Crime_Index`	Numeric	Area crime index score
`Air_Quality_Index`	Numeric	AQI score for the locality
`Neighborhood_Growth_Rate_%`	Numeric	Percentage neighbourhood growth rate
`Price_per_SqFt`	Numeric	Market price per square foot (INR)
`Annual_Property_Tax`	Numeric	Annual tax liability (INR)
`Estimated_Rental_Yield_%`	Numeric	Estimated rental return percentage
`Neighborhood`	Categorical	One of: Downtown, IT Hub, Industrial, Residential, Suburban

Preprocessing Pipeline

1. **Missing value imputation**: Median imputation applied to all numeric columns.
2. **Ordinal encoding**: `Furnishing_Status` encoded as 0, 1, 2 preserving order.
3. **One-hot encoding**: `Neighborhood` expanded into four binary columns (`Neighborhood_IT Hub`, `Neighborhood_Industrial`, `Neighborhood_Residential`, `Neighborhood_Suburba` with `Downtown` as the reference category.
4. **Outlier clipping**: IQR-based clipping applied to numeric features to remove extreme values.
5. **Feature scaling**: `MinMaxScaler` fitted on the training set and saved as `joblib` artifacts for consistent inference-time transformation.

Milestone 1: Machine Learning Models

Model Selection

Two separate XGBoost models were trained: one for regression (predicting continuous market price) and one for multi-class classification (predicting investment grade). XGBoost was selected because it consistently outperforms linear and tree-based baselines on medium-sized structured tabular datasets by capturing non-linear interactions between features without requiring deep learning infrastructure.

Hyperparameter Configuration

Table 3: Hyperparameter configuration for both XGBoost models

Parameter	Regression	Classification
`n_estimators`	200	200
`learning_rate`	0.05	0.05
`max_depth`	5	4
`subsample`	0.8	0.8
`colsample_bytree`	0.8	0.8
Scaling	MinMaxScaler	MinMaxScaler

Model Performance Results

Regression Model — Current Market Price Prediction

R^2 Score	**0.9483**
Mean Absolute Error (MAE)	**Rs 550,851**
Root Mean Squared Error (RMSE)	**Rs 1,010,636**

An R^2 of 0.9483 indicates the regression model explains approximately 94.8% of the variance in property prices, demonstrating strong predictive fidelity on the held-out test set.

Classification Model — Investment Grade Prediction

Accuracy	**97.50%**
Weighted F1	**0.97**
Classes	0 (Low), 1 (Medium), 2 (High)

The classification model achieves 97.5% accuracy across three investment grade classes, with a weighted F1 of 0.97 confirming strong performance across all class imbalances.

Key Performance Drivers

The strongest predictive features identified by model analysis include:

- `Price_per_SqFt` — the single most influential feature, as it directly encodes local market pricing.
- `Total_Square_Footage` and `Neighborhood_Growth_Rate_%` — capture size and market trajectory.

- `Distance_to_City_Center_km` and `Proximity_to_Public_Transport_km` — encode accessibility premium.
- `Estimated_Rental_Yield_%` — directly informs investment-grade classification.
- `Crime_Index` and `Air_Quality_Index` — encode neighbourhood quality signals.

Saved Artifacts

All four model artifacts are serialised with `joblib` and stored in `models/`:

- `xgb_regression_model.joblib`
- `regression_scaler.joblib`
- `xgb_classification_model.joblib`
- `classification_scaler.joblib`

The scalers store `feature_names_in_` metadata, which the application validates at startup to ensure that the inference-time feature order exactly matches the training-time order.

Milestone 2: Agentic Real Estate Advisory System

System Overview

Milestone 2 extends the ML prediction pipeline into a fully agentic workflow orchestrated by LangGraph. The system accepts input through three paths (natural-language prompt, manual form entry, and CSV batch upload), passes inputs through typed LangGraph nodes, and produces a structured advisory report comprising a property valuation summary, comparable property analysis, an investment recommendation, and a financial disclaimer.

Technology Stack and Design Justification

Table 4: Technology choices and justification

Component	Choice	Justification
Agentic framework	LangGraph 1.1.6	Named nodes with shared TypedDict state; workflow visible and debuggable as a graph rather than implicit function chains.
LLM provider	Groq (llama-3.3-70b-versatile)	Free-tier hosted inference; fastest available open-weight model for prompt extraction and constrained explanation generation.
ML backbone	XGBoost 3.2.0	Strongest tabular baseline; deterministic and fast at inference time.
UI framework	Streamlit 1.54.0	Supports session state, dialogs, file upload, and metrics natively; simple to deploy on Streamlit Community Cloud.
Email notification	smtplib / SMTP	Standard library; no paid API or phone verification required.
Retrieval	Pandas CSV filter	Knowledge base is structured tabular data; column-based filtering by neighbourhood and square footage is more precise and interpretable than embedding-based semantic similarity. A vector database (FAISS/Chroma) is appropriate for unstructured text corpora, not for structured property tables.
Hosting	Streamlit Community Cloud	Free-tier; satisfies the mandatory public deployment requirement.

LangGraph State and Graph Architecture

The entire agentic workflow is defined in `app/property_graph.py`. A shared `TypedDict` state object is passed between every node:

```python
class PropertyWorkflowState(TypedDict, total=False):
    mode: str                        # "prompt" | "manual" | "csv"
    prompt: str                      # raw user text
    context: dict[str, Any]          # loaded models, scalers, defaults
    secrets: Any                     # Streamlit secrets
    input_settings: dict[str, Any]   # Groq API key and model
    explanation_settings: dict[str, Any]
    email_settings: dict[str, Any]
    numeric_inputs: dict[str, Any]   # extracted property fields
    furnishing: str
    neighborhood: str
    flow: dict[str, Any]             # assembled field + source map
```

```
    result: dict[str, Any]          # price, grade, confidence, probs
    comparables: list[dict]         # comparable properties from CSV
    explanation: str                # Groq-generated explanation
    recipient: str
    email_sent: bool
    input_df: Any                   # uploaded DataFrame
    output_df: Any                  # batch prediction output
    review_ready: bool
```

Listing 1: PropertyWorkflowState TypedDict definition

Six separate `StateGraph` instances are compiled and cached at module load:

Table 5: Compiled LangGraph graphs and their node sequences

Graph	Node sequence
PROMPT_REVIEW_GRAPH	START → input_node → review_node → END
CONFIRMED_PREDICTION_GRAPH	START → prediction_node → explanation_node → END
MANUAL_PREDICTION_GRAPH	START → input_node → prediction_node → explanation_node → END
CSV_PREDICTION_GRAPH	START → csv_prediction_node → END
RESULT_EMAIL_GRAPH	START → notification_node → END
CSV_EMAIL_GRAPH	START → notification_node → END

Node-by-Node Description

Table 6: LangGraph node responsibilities and state keys

Node	File	Reads	Writes
input_node (prompt)	input_nodes.py	prompt, context, input_settings	mode, flow
review_node	property_graph.py	flow	review_ready
input_node (manual)	property_graph.py	numeric_inputs, furnishing, neighborhood	mode, flow
prediction_node	prediction_nodes.py	flow, context	result, comparables
explanation_node	explanation_nodes.py	flow, result, comparables, explanation_settings	explanation
csv_prediction_node	prediction_nodes.py	input_df, context	mode, output_df
notification_node	notification_nodes.py	recipient, result, flow, explanation, comparables, email_settings	email_sent

Three Input Paths

Path 1 — Prompt Agent

The user describes a property in free-form English. The Input Node sends the text to Groq with a schema-locked system prompt (temperature = 0, JSON response format, allowlisted field names). Groq returns a structured JSON object. Any field not mentioned in the prompt is filled with the training-data median. The Review Node marks the state as ready and Streamlit opens a popup showing each value with a *Prompt / Default / Edited* source label. After the user confirms (or edits), the Prediction and Explanation Nodes run.

Path 2 — Manual Input

The user enters all fields directly in a form. The Manual Input Node wraps these into the same `flow` format used by the Prompt path, then reuses the identical Prediction, Explanation, and Notification Nodes.

Path 3 — CSV Batch Upload

The user uploads a CSV file. The CSV Prediction Node validates that the file contains either raw user-facing columns or already-encoded feature columns, builds the feature matrix, runs both XGBoost models across all rows, and returns an augmented DataFrame with `Predicted_Price_INR`, `Predicted_Investment_Grade`, and per-class probabilities. Results are shown as a table and can be downloaded or emailed.

Comparable Property Retrieval

The Prediction Node calls `find_comparable_properties()` from `prediction_nodes.py`, which:

1. Filters the training dataset for rows matching the same `Neighborhood`.
2. Further filters to properties within ±25% of the query's `Total_Square_Footage`.
3. Sorts by absolute distance from the predicted price.
4. Returns the top 5 matches as a list of dictionaries.

These comparable rows are:

- Displayed as a table under the prediction result in the Streamlit UI.
- Injected into the Groq explanation prompt so the LLM can comment on whether the predicted price is above or below comparable properties.
- Included as a section in the HTML email report.

Prompting Strategy and Anti-Hallucination Guardrails

Five explicit guardrails are implemented to reduce hallucination risk:

Guardrail 1 — Schema-Locked Input Extraction

The Input Node system prompt instructs Groq to return only JSON with three keys: `numeric_inputs`, `furnishing_status`, and `neighborhood`. The `numeric_inputs` dictionary is validated against an allowlist of 13 field names. Unknown keys are silently discarded. Extraction

runs at temperature = 0 with `response_format: {type: json_object}` to enforce deterministic structured output.

Guardrail 2 — Human Review Before Prediction

The Prompt Agent never predicts immediately after extraction. It pauses at the Review Node, which triggers a Streamlit dialog popup. Each extracted value is labelled by source (*Prompt*, *Default*, or *Edited*). The user can modify any value before the Prediction Node runs, preventing hidden defaults from silently propagating into model inputs.

Guardrail 3 — Grounded Explanation Prompt

The Explanation Node sends Groq a compact JSON object containing only confirmed inputs, predicted price, advisory class, confidence, and the comparable properties retrieved from training data. The system prompt explicitly instructs the model to:

- not invent outside market facts, legal claims, or financial advice,
- use the phrase "Based on the model output" for uncertain claims,
- not cite exact feature importance values.

Guardrail 4 — Output Format Constraint

The Explanation Node requires exactly four labelled bullets: **Summary**, **Market Context**, **Recommendation**, and **Risk Warning**. Fixing the output structure reduces the surface area for hallucinated content and keeps the advisory report consistent across all inputs.

Guardrail 5 — Safe Fallbacks

If `GROQ_API_KEY` is missing, the Input Node switches to a local rule-based regex parser so the system degrades gracefully. If the Explanation API call fails, a deterministic local fallback explanation is shown. Neither path crashes the UI or returns an empty advisory report.

Structured Advisory Output

The final advisory output satisfies all four structured output requirements from the rubric:

Table 7: Structured output sections and implementation

Section	Status	Implementation
Summary (Property Valuation & Market View)	Implemented	Advisory card showing predicted price, investment grade label, and confidence percentage. Colour-coded by grade (Avoid / Hold / Buy).
Comps (Comparable Property Analysis)	Implemented	Table of top-5 comparable properties from training data, filtered by neighbourhood and square footage. Shown in UI and email.
Action (Buy/Invest Recommendation)	Implemented	Investment grade mapped to advisory label: 0 = Avoid, 1 = Hold, 2 = Buy. Per-grade probability table shown alongside.
Disclaimer (Financial/Legal Notices)	Implemented	"This application is a student project for educational purposes only. It does not constitute financial, legal, or investment advice." Shown on About page and in email footer.

Email Notification

The Notification Node sends a formatted HTML email via SMTP when the user provides a recipient address and clicks Send. The email contains:

- Three metric cards: Predicted Price, Advisory Recommendation, Confidence.
- A property summary table with source labels (Prompt / Default / Edited).
- A comparable properties table.
- The Groq-generated explanation with key terms highlighted.
- A financial disclaimer footer.

CSV batch predictions are emailed as a `.csv` attachment.

System Architecture

Architecture Diagram

Figure 1 shows the complete end-to-end system architecture, from the three user-facing input paths through the LangGraph agentic nodes to the final advisory output and optional email notification.

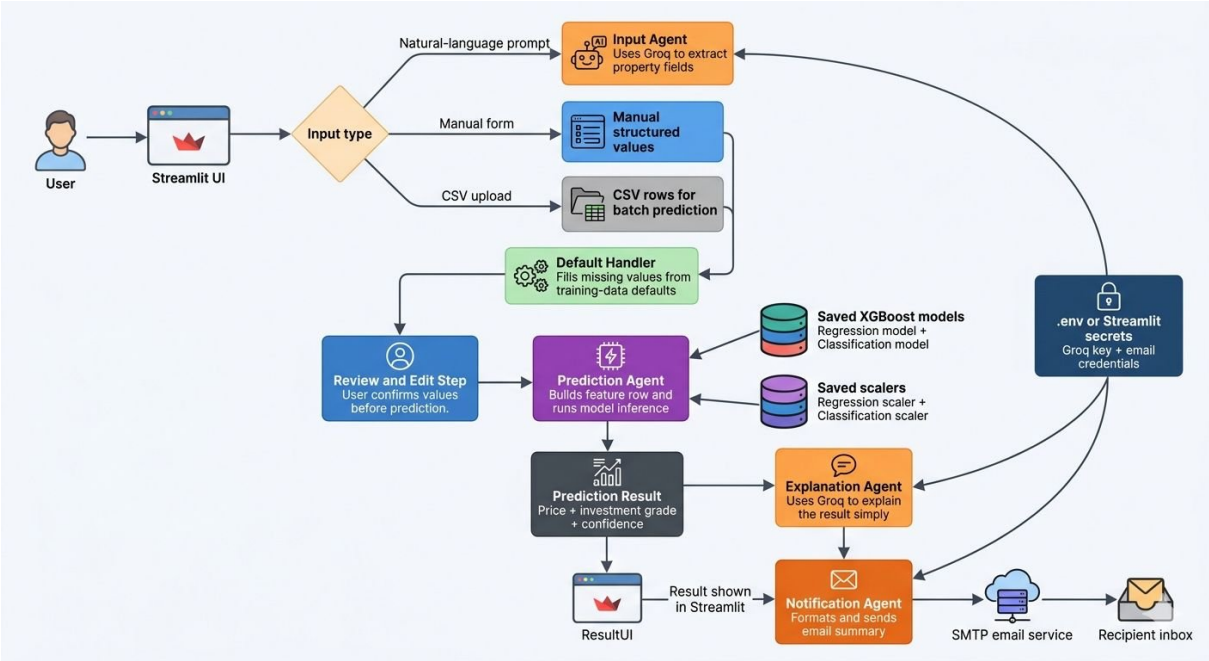

Figure 1: End-to-end system architecture: three input paths (natural-language prompt, manual form, CSV upload), LangGraph agentic nodes, XGBoost inference with saved model artifacts, and advisory output delivery via Streamlit UI and SMTP email.

Repository Structure

The repository is organised to keep concerns cleanly separated: the `app/` directory holds all runtime code, `data/` holds raw and processed datasets, `models/` holds serialised model artifacts, and `notebooks/` holds the offline training notebook. Configuration, secrets, and documentation live at the project root.

```
property-price-prediction/
|-- app/
|    |-- streamlit_app.py        # Application entry point and page router
|    |-- property_graph.py       # LangGraph state, nodes, compiled graphs
|    |-- input_nodes.py          # Groq extraction + rule-based fallback
|    |-- prediction_nodes.py     # Feature engineering, ML inference, comps
|    |-- explanation_nodes.py    # Grounded Groq explanation
|    |-- notification_nodes.py   # HTML email + CSV attachment delivery
|    |-- pages.py                # Streamlit UI pages and dialogs
|    |-- config.py               # Paths, constants, feature lists
|    `-- styles.py               # CSS styling
|-- .streamlit/
|    `-- config.toml             # Streamlit theme configuration
|-- data/
|    |-- raw/real_estate_raw.csv
|    `-- processed/real_estate_clean.csv
|-- models/
|    |-- xgb_regression_model.joblib
|    |-- regression_scaler.joblib
|    |-- xgb_classification_model.joblib
|    `-- classification_scaler.joblib
|-- notebooks/
|    `-- training_colab.ipynb    # Full training and evaluation notebook
|-- report/
```

```
|    |-- property_price_prediction_report.pdf
|    `-- property_price_prediction_report.tex
|-- system_architecture.png     # End-to-end architecture diagram
|-- agent_workflow.md           # Detailed agentic flow documentation
|-- .env.example                # Placeholder API key configuration
|-- requirements.txt
|-- runtime.txt
`-- README.md
```

Listing 2: Project repository structure

End-to-End Data Flow

1. **User Input** — entered via Prompt Agent, Manual Input form, or CSV upload.
2. **Input Node** — Groq extracts structured fields from natural-language input; defaults fill missing values.
3. **Review Node** — Streamlit dialog shows extracted values by source; user confirms or edits.
4. **Prediction Node** — feature row constructed, scaled, passed to both XGBoost models; comparable properties retrieved from training CSV.
5. **Explanation Node** — confirmed inputs, model output, and comparables sent to Groq; four-bullet advisory explanation returned.
6. **UI Render** — advisory card, property summary, comparable table, probability table, and explanation rendered.
7. **Notification Node** — optional: HTML email with full report sent via SMTP.

Streamlit Pages

Table 8: Application pages and their functions

Page	Function
Prompt Agent	Runs the full LangGraph flow: Groq extraction → review popup → prediction → explanation → email.
CSV Upload	Batch prediction for uploaded CSV; download or email the output file.
Manual Input	Direct field entry form; reuses the same Prediction, Explanation, and Notification nodes.
About	Summarises the LangGraph flow, model metrics, advisory label mapping, and financial disclaimer.

Deployment and Reproducibility

Public Deployment

The application is publicly hosted on Streamlit Community Cloud at:

https://property-price-prediction-real-estate.streamlit.app/

Localhost-only demonstrations are not accepted per project requirements. This deployment satisfies the mandatory public hosting criterion.

Local Setup

```
# 1. Clone repository
git clone https://github.com/Jag2007/property-price-prediction.git
cd property-price-prediction

# 2. Create virtual environment
python3 -m venv venv && source venv/bin/activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Configure secrets (copy from .env.example)
cp .env.example .env
# Edit .env with your GROQ_API_KEY and email credentials

# 5. Run the application
streamlit run app/streamlit_app.py
```

Listing 3: Local installation and run commands

Environment Variables

Table 9: Required environment variables

Variable	Required	Purpose
GROQ_API_KEY	Yes (Prompt/Explain)	Authenticates Groq API calls
GROQ_MODEL	Optional	Defaults to llama-3.3-70b-versatile
EMAIL_SMTP_HOST	Email only	SMTP host (e.g. smtp.gmail.com)
EMAIL_SMTP_PORT	Email only	SMTP port (587)
EMAIL_SENDER	Email only	Sender email address
EMAIL_APP_PASSWORD	Email only	Gmail app password
EMAIL_FROM_NAME	Optional	Display name in sent emails

Key Dependencies

Table 10: Core library versions

Library	Version
streamlit	1.54.0
langgraph	1.1.6
xgboost	3.2.0
scikit-learn	1.6.1
pandas	2.3.3
numpy	2.2.6
joblib	1.5.2

Results and Qualitative Examples

Quantitative Model Performance

Table 11: Final model evaluation metrics on held-out test set

Model	Metric	Value
XGBoost Regression	R^2 Score	0.9483
	MAE	Rs 550,851
	RMSE	Rs 1,010,636
XGBoost Classification	Accuracy	97.50%
	Weighted F1	0.97

Qualitative Prompt Agent Example

User input (Prompt Agent):
"1450 sqft 3 BHK, 2 bathrooms, 6 years old, 8th floor, semi furnished, IT Hub, 4.5 km from city center, 0.7 km from metro, crime index 32, AQI 88, growth 9%, price per sqft 7200, annual tax 85000, rental yield 4.2%"

System output:

Predicted Price	Rs 10,420,000
Advisory Recommendation	**Buy** (Grade 2)
Confidence	91.4%

Groq explanation (Summary bullet): The property is priced in line with 4 comparable IT Hub properties of similar size. The high rental yield (4.2%) and strong neighbourhood growth (9%) are the primary drivers of the Buy recommendation. The low crime index and proximity to public transport further support a high investment grade.

Comparable properties retrieved: 5 properties from IT Hub neighbourhood within 25% of 1450 sqft, price range Rs 9.8M – Rs 11.1M.

Advisory Label Mapping

Table 12: Investment grade to advisory label mapping

Model Class	Advisory Label	Meaning
0	Avoid	Weakest investment class in the trained model.
1	Hold	Moderate class; review details before committing.
2	Buy	Strongest investment class in the trained model.

Responsible AI and Ethical Considerations

Financial Disclaimer

This application is a student project for educational purposes only. It does not constitute financial, legal, or investment advice. All predictions are generated by machine learning models trained on synthetic data and should not be used as the sole basis for any real-world property investment decision.

Data Transparency

Every value presented in the advisory report is labelled by its origin:

- **Prompt** — extracted directly from the user's input text.
- **Default** — filled automatically from the training-data median.
- **Edited** — modified by the user in the review popup.

This labelling prevents hidden defaults from silently entering model inputs and gives users full visibility into how the advisory report was generated.

Hallucination Mitigation

The five guardrails described in Section 4.5 collectively ensure the LLM component generates only grounded, bounded outputs. The explanation is constrained to four fixed bullets, forbidden from inventing market facts or giving financial advice, and required to ground every claim in the confirmed inputs and retrieved comparable properties.

Model Limitations

- The models were trained on synthetic data and may not generalise to real-world property markets.
- Prompt extraction quality depends on the clarity of the user's text and on Groq API availability.

- The comparable-property retrieval draws from the same synthetic training dataset, not from live market listings.
- Investment grade labels (Avoid / Hold / Buy) reflect model output classes and carry no legal or professional standing.

Conclusion

This project demonstrates a complete progression from classical ML-based price prediction to a LangGraph-orchestrated agentic advisory system. The key contributions are:

1. A high-accuracy XGBoost prediction pipeline achieving $R^2 = 0.9483$ for price regression and 97.5% accuracy for investment grade classification.
2. A genuine LangGraph implementation with a shared `TypedDict` state, six compiled `StateGraph` instances, and proper node-edge wiring — not a superficial renaming of sequential function calls.
3. Five documented anti-hallucination guardrails applied at every LLM interaction point.
4. A structured advisory output satisfying all four rubric requirements: Summary, Comps, Action, and Disclaimer.
5. A publicly deployed application on Streamlit Community Cloud with three input modes, email notification, and a reproducible local setup documented through `.env.example` and step-by-step README instructions.
6. Deterministic comparable-property retrieval that grounds LLM explanations in real training-data evidence, with a documented design justification for why semantic vector search was not appropriate for this use case.

The project validates that a free-tier technology stack (Groq, LangGraph, XGBoost, Streamlit) is sufficient to build and deploy a production-quality agentic real estate advisory system.

References

1. LangGraph Documentation. *LangGraph: Build stateful, multi-actor applications with LLMs*. LangChain Inc., 2024. https://langchain-ai.github.io/langgraph/
2. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
3. Groq Documentation. *Groq API Reference*. Groq Inc., 2024. https://console.groq.com/docs/
4. Streamlit Documentation. *Streamlit: The fastest way to build and share data apps*. Snowflake Inc., 2024. https://docs.streamlit.io/

5. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
6. GitHub Repository: https://github.com/Jag2007/property-price-prediction
7. Live Application: https://property-price-prediction-real-estate.streamlit.app/